

AI ASSISTANT CODING

ASSIGNMENT – 6.5

NAME : G.RISHITHA

HT.NO : 2303A52149

BATCH : 35

Task Description -1 (AI-Based Code Completion for Conditional Eligibility Check) :

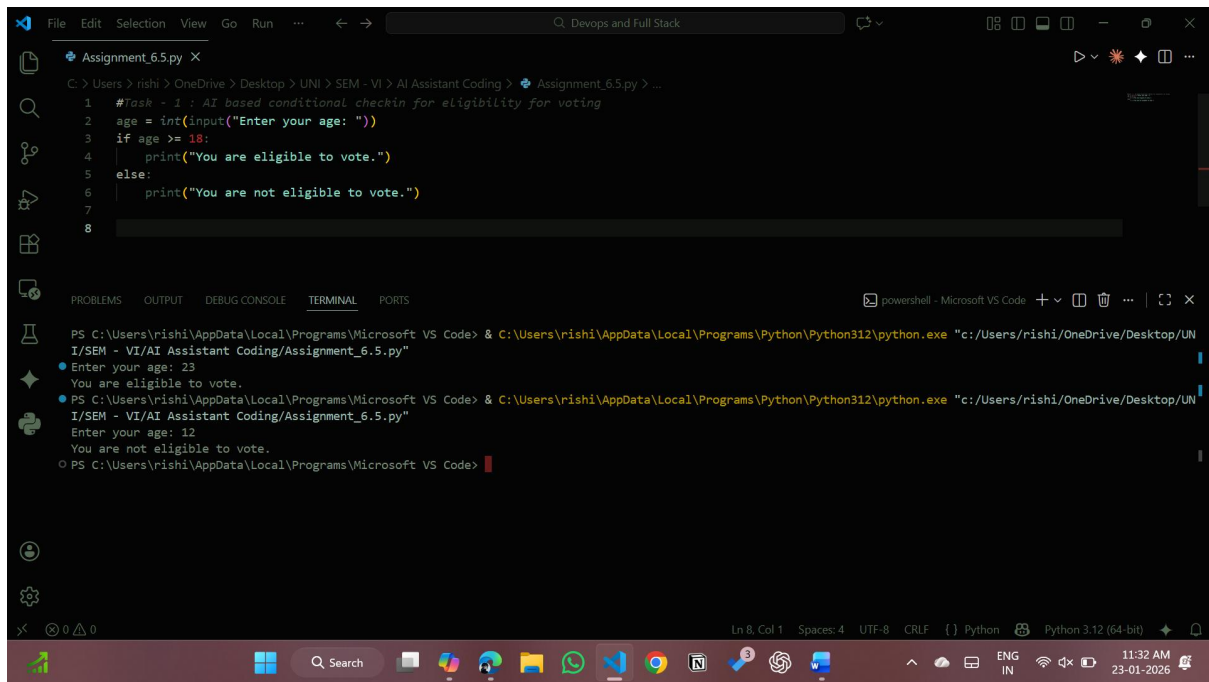
Task: Use an AI tool to generate eligibility logic.

Prompt: “Generate Python code to check voting eligibility based on age and citizenship.”

Expected Output:

- AI-generated conditional logic.
- Correct eligibility decisions.
- Explanation of conditions.

CODE & OUTPUT:



The screenshot shows a Microsoft VS Code editor window with a Python file named `Assignment_6.5.py`. The code is a simple conditional check for voting eligibility based on age. The terminal window at the bottom shows the execution of the script using Python 3.12. The user enters '23' and '12' as input, and the program outputs 'You are eligible to vote.' and 'You are not eligible to vote.' respectively.

```
1 #Task - 1 : AI based conditional checkin for eligibility for voting
2 age = int(input("Enter your age: "))
3 if age >= 18:
4     print("You are eligible to vote.")
5 else:
6     print("You are not eligible to vote.")
7
8
```

Terminal Output:

```
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment_6.5.py"
Enter your age: 23
You are eligible to vote.
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment_6.5.py"
Enter your age: 12
You are not eligible to vote.
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code>
```

Task Description – 2 (AI-Based Code Completion for Loop-Based String Processing) :

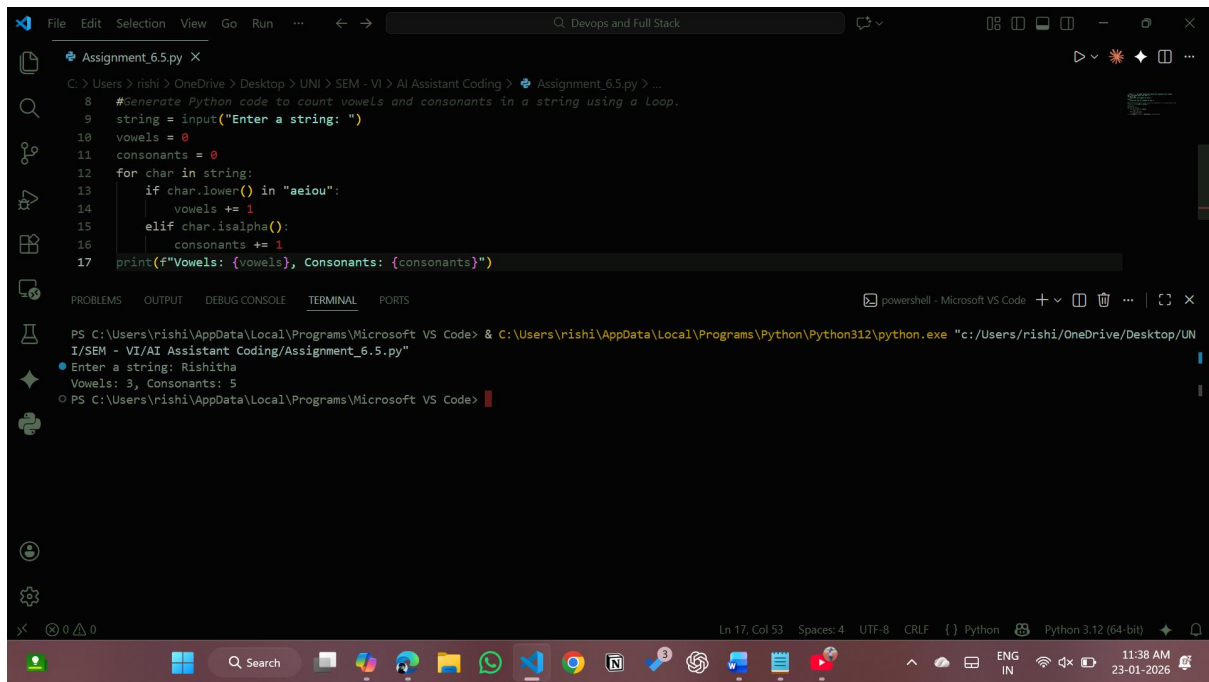
Task: Use an AI tool to process strings using loops.

Prompt: “Generate Python code to count vowels and consonants in a string using a loop.”

Expected Output:

- AI-generated string processing logic.
- Correct counts.
- Output verification.

CODE & OUTPUT:



```
8 #Generate Python code to count vowels and consonants in a string using a loop.
9 string = input("Enter a string: ")
10 vowels = 0
11 consonants = 0
12 for char in string:
13     if char.lower() in "aeiou":
14         vowels += 1
15     elif char.isalpha():
16         consonants += 1
17 print(f"Vowels: {vowels}, Consonants: {consonants}")
```

PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment_6.5.py"

Enter a string: Rishitha

Vowels: 3, Consonants: 5

PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code>

Task Description - 3 (AI-Assisted Code Completion Reflection Task) :

Task: Use an AI tool to generate a complete program using classes, loops, and conditionals.

Prompt: “Generate a Python program for a library management system using classes, loops, and conditional statements.”

Expected Output:

- Complete AI-generated program.
- Review of AI suggestions quality.
- Short reflection on AI-assisted coding experience.

CODE:

```
File Edit Selection View Go Run ... DevOps and Full Stack
Assignment_6.5.py X
C:\Users\rishi> OneDrive > Desktop > UNI > SEM - VI > AI Assistant Coding > Assignment_6.5.py > ...
19 #generate a Python program for a library management system using classes, loops, and conditional statements.
20 class Library:
21     def __init__(self, books):
22         self.books = books
23
24     def display_books(self):
25         print("Available Books:")
26         for book in self.books:
27             print(book)
28
29     def borrow_book(self, book_name):
30         if book_name in self.books:
31             self.books.remove(book_name)
32             print(f"You have borrowed '{book_name}'.")
33         else:
34             print(f"Sorry, '{book_name}' is not available.")
35
36     def return_book(self, book_name):
37         self.books.append(book_name)
38         print(f"You have returned '{book_name}'.")
39
40 library = Library(["1984", "To Kill a Mockingbird", "The Great Gatsby"])
41 while True:
42     print("\nLibrary Menu:")
43     print("1. Display Books")
44     print("2. Borrow a Book")
45     print("3. Return a Book")
46     print("4. Exit")
47     choice = input("Enter your choice: ")
```

OUTPUT:

```
File Edit Selection View Go Run ... DevOps and Full Stack
Assignment_6.5.py X
C:\Users\rishi> OneDrive > Desktop > UNI > SEM - VI > AI Assistant Coding > Assignment_6.5.py > ...
20 class Library:
24     def display_books(self):
27         print(book)
28
29     def borrow_book(self, book_name):
30         if book_name in self.books:
31             self.books.remove(book_name)
32             print(f"You have borrowed '{book_name}'.")
33         else:
34             print(f"Sorry, '{book_name}' is not available.")
35
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
python - Microsoft VS Code
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment_6.5.py"

Library Menu:
1. Display Books
2. Borrow a Book
3. Return a Book
4. Exit
Enter your choice: 1984

Library Menu:
1. Display Books
2. Borrow a Book
3. Return a Book
4. Exit
Enter your choice: █
```

Task Description - 4 (AI-Assisted Code Completion for Class-Based Attendance System) :

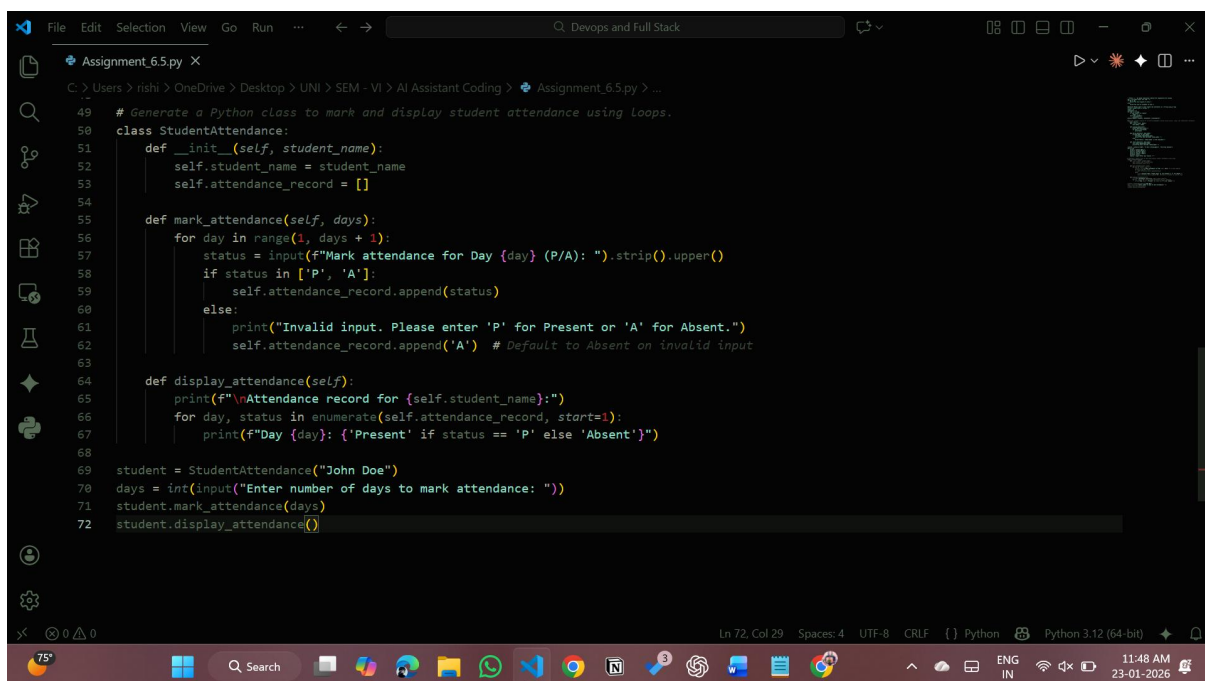
Task: Use an AI tool to generate an attendance management class.

Prompt: “Generate a Python class to mark and display student attendance using loops.”

Expected Output:

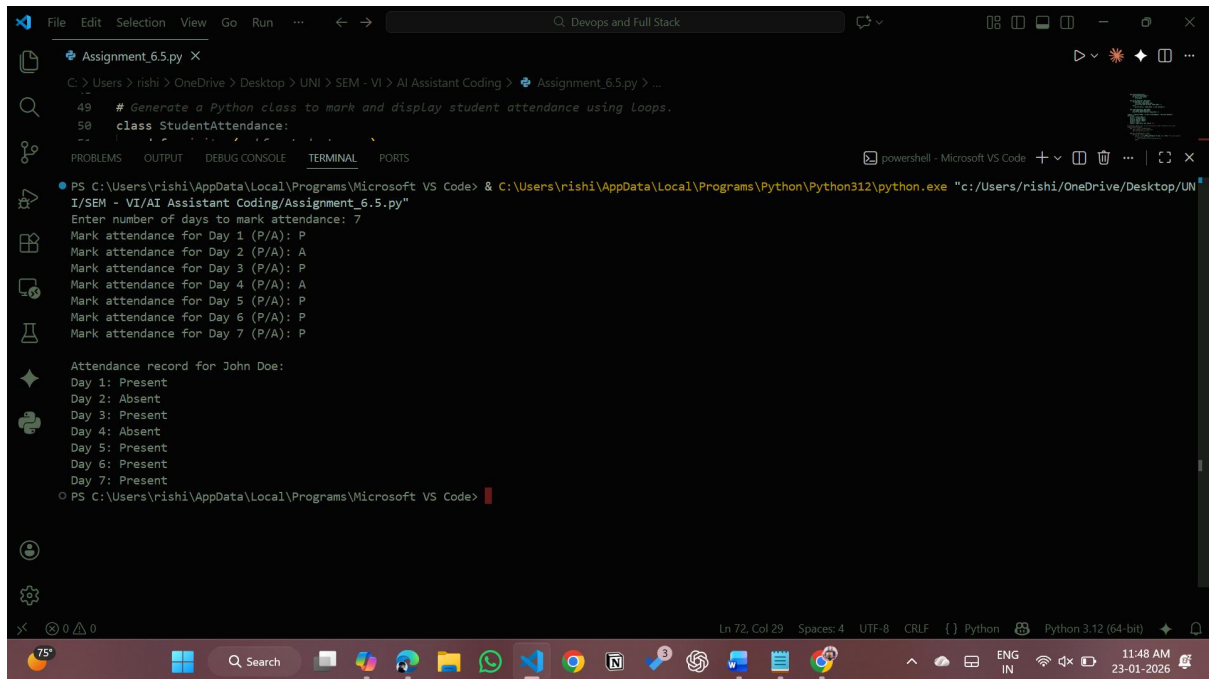
- AI-generated attendance logic.
- Correct display of attendance.
- Test cases.

CODE:



```
49 # Generate a Python class to mark and display student attendance using Loops.
50 class StudentAttendance:
51     def __init__(self, student_name):
52         self.student_name = student_name
53         self.attendance_record = []
54
55     def mark_attendance(self, days):
56         for day in range(1, days + 1):
57             status = input(f"Mark attendance for Day {day} (P/A): ").strip().upper()
58             if status in ['P', 'A']:
59                 self.attendance_record.append(status)
60             else:
61                 print("Invalid input. Please enter 'P' for Present or 'A' for Absent.")
62                 self.attendance_record.append('A') # Default to Absent on invalid input
63
64     def display_attendance(self):
65         print(f"\nAttendance record for {self.student_name}:")
66         for day, status in enumerate(self.attendance_record, start=1):
67             print(f"Day {day}: {'Present' if status == 'P' else 'Absent'}")
68
69 student = StudentAttendance("John Doe")
70 days = int(input("Enter number of days to mark attendance: "))
71 student.mark_attendance(days)
72 student.display_attendance()
```

OUTPUT:



```
Assignment_6.5.py X
C:\Users\rishi> OneDrive\ Desktop> UNI> SEM - VI> AI Assistant Coding> Assignment_6.5.py > ...
49 # Generate a Python class to mark and display student attendance using Loops.
50 class StudentAttendance:
--
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment_6.5.py"
Enter number of days to mark attendance: 7
Mark attendance for Day 1 (P/A): P
Mark attendance for Day 2 (P/A): A
Mark attendance for Day 3 (P/A): P
Mark attendance for Day 4 (P/A): A
Mark attendance for Day 5 (P/A): P
Mark attendance for Day 6 (P/A): P
Mark attendance for Day 7 (P/A): P

Attendance record for John Doe:
Day 1: Present
Day 2: Absent
Day 3: Present
Day 4: Absent
Day 5: Present
Day 6: Present
Day 7: Present
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code>
```

Task Description - 5 (AI-Based Code Completion for Conditional Menu Navigation) :

Task: Use an AI tool to complete a navigation menu.

Prompt: “Generate a Python program using loops and conditionals to simulate an ATM menu.”

Expected Output:

- AI-generated menu logic.
- Correct option handling.
- Output verification.

CODE:

```
Assignment_6.5.py X
C:\Users> rishi> OneDrive> Desktop> UNI> SEM - VI> AI Assistant Coding> Assignment_6.5.py> ...

74 # Generate a Python program using Loops and conditionals to simulate an ATM menu.
75 class ATM:
76     def __init__(self, balance=1000):
77         self.balance = balance
78     def display_menu(self):
79         print("\nATM Menu:")
80         print("1. Check Balance")
81         print("2. Deposit Money")
82         print("3. Withdraw Money")
83         print("4. Exit")
84     def check_balance(self):
85         print(f"Your balance is ${self.balance}")
86     def deposit_money(self, amount):
87         self.balance += amount
88         print(f"${amount} deposited. New balance is ${self.balance}")
89     def withdraw_money(self, amount):
90         if amount > self.balance:
91             print("Insufficient funds.")
92         else:
93             self.balance -= amount
94             print(f"${amount} withdrawn. New balance is ${self.balance}")
95 atm = ATM()
96 while True:
97     atm.display_menu()
98     choice = input("Enter your choice: ")
99     if choice == '1':
100         atm.check_balance()
101     elif choice == '2':
102         amount = float(input("Enter amount to deposit: "))
```

```
Assignment_6.5.py X
C:\Users> rishi> OneDrive> Desktop> UNI> SEM - VI> AI Assistant Coding> Assignment_6.5.py> ...

75 class ATM:
76     def deposit_money(self, amount):
77         print(f"${amount} deposited. New balance is ${self.balance}")
78     def withdraw_money(self, amount):
79         if amount > self.balance:
80             print("Insufficient funds.")
81         else:
82             self.balance -= amount
83             print(f"${amount} withdrawn. New balance is ${self.balance}")
84 atm = ATM()
85 while True:
86     atm.display_menu()
87     choice = input("Enter your choice: ")
88     if choice == '1':
89         atm.check_balance()
90     elif choice == '2':
91         amount = float(input("Enter amount to deposit: "))
92         atm.deposit_money(amount)
93     elif choice == '3':
94         amount = float(input("Enter amount to withdraw: "))
95         atm.withdraw_money(amount)
96     elif choice == '4':
97         print("Thank you for using the ATM. Goodbye!")
98         break
99     else:
100         print("Invalid choice. Please try again.")
```

OUTPUT:

```
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment_6.5.py"

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice: 1
Your balance is $1000

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice: 2
Enter amount to deposit: 4500
$4500.0 deposited. New balance is $5500.0

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice: 3
Enter amount to withdraw: 2000
$2000.0 withdrawn. New balance is $3500.0

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
```

```
PS C:\Users\rishi\AppData\Local\Programs\Microsoft VS Code> & C:\Users\rishi\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/rishi/OneDrive/Desktop/UNI/SEM - VI/AI Assistant Coding/Assignment_6.5.py"

3. Withdraw Money
4. Exit
Enter your choice: 2
Enter amount to deposit: 4500
$4500.0 deposited. New balance is $5500.0

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice: 3
Enter amount to withdraw: 2000
$2000.0 withdrawn. New balance is $3500.0

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice: 1
Your balance is $3500.0

ATM Menu:
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter your choice: 4
Thank you for using the ATM. Goodbye!
```